

Error Feedback Granularity for LLM Code Repair: An Empirical Study at the 7B Scale

Anonymous Authors¹

Abstract

This paper investigates the effect of error feedback granularity on LLM code repair success. In a controlled experiment using CodeLlama-7B-Instruct on 304 MBPP runtime error cases, five granularity levels were compared: pass/fail only (G0), error type (G1), error message (G2), error with line number (G3), and full stack trace (G4). One-way ANOVA indicates that granularity has a statistically significant effect on repair success ($F=23.89$, $p=3.5e-19$, $\eta^2=0.059$). Contrary to the initial hypothesis that intermediate granularity would be optimal, the results show that minimal feedback (G0, G1) achieves higher repair success rates (41.8% and 40.8%) than detailed feedback (G2: 18.4%, G3: 16.8%, G4: 22.7%). The difference between G0 and G3 is 25 percentage points in favor of G0 (McNemar $p=5.2e-22$). These findings are specific to CodeLlama-7B-Instruct on MBPP and should not be extrapolated to larger models or different benchmarks without further empirical validation.

1. Introduction

When LLM-generated code fails a test, one approach to repair involves providing the model with error feedback and requesting a fix. Prior work on self-debugging and iterative code repair has used various forms of error feedback, ranging from simple pass/fail signals to detailed execution traces with variable states. However, the effect of feedback granularity on repair success has not been systematically studied.

¹Anonymous Institution, Anonymous City, Anonymous Region, Anonymous Country. Correspondence to: Anonymous Author <anonymous@anonymous.org>

Preliminary work. Under review by the International Conference on Machine Learning (ICML). Do not distribute.

This work addresses the following research question: Does the granularity of error feedback affect LLM code repair success, and if so, how?

1.0.1. 1.1 BACKGROUND

The self-debugging paradigm, established by Chen et al. (2023), demonstrated that LLMs can improve their own code using execution feedback, achieving reported improvements on benchmarks such as MBPP. Subsequent work has extended this approach with execution traces (Bouzenia et al., 2024), reinforcement learning from execution feedback (Gehring et al., 2024), and multi-agent frameworks (?). These approaches have used different levels of error detail, but none have systematically compared granularity levels as the primary variable.

1.0.2. 1.2 RESEARCH GAP

Prior work varies feedback granularity incidentally rather than systematically. Self-Debug uses error messages (approximately G2-level feedback), TraceFixer uses full stack traces (G4-level), and DynaFix adds variable states. No prior controlled comparison exists that isolates granularity as the sole independent variable.

1.0.3. 1.3 HYPOTHESES

This study tested the following hypotheses:

- **H-E1 (Foundation):** Runtime errors with localizable stack traces constitute at least 30% of LLM code generation failures on MBPP.
- **H-M1 (Mechanism):** Error feedback granularity has a statistically significant effect on repair success rate (ANOVA $p < 0.05$).
- **H-M2 (Direction):** G3 (error with line number) achieves at least 10 percentage points higher repair success than G0 (pass/fail only).
- **H-M3 (Non-monotonicity):** G4 (full stack

trace) does not significantly outperform G3 (difference within 2%).

The directional hypotheses (H-M2, H-M3) were based on an "attention window hypothesis" predicting that intermediate granularity would focus model attention optimally.

1.0.4. 1.4 SUMMARY OF FINDINGS

H-E1 and H-M1 were supported by the data. H-M2 and H-M3 were not supported; the observed pattern was opposite to the predicted direction for H-M2, and G4 significantly outperformed G3 contrary to H-M3.

2. Related Work

2.0.1. 2.1 LLM SELF-REPAIR

Chen et al. (2023) introduced self-debugging, where LLMs explain their code to identify mistakes. This work reported 12% improvement on MBPP using error message feedback. Jiang et al. (2024) extended this with chain-of-explanation before refinement. Li et al. (2024) applied tree search over the repair space.

2.0.2. 2.2 EXECUTION TRACE-BASED REPAIR

TraceFixer (Bouzenia et al., 2024) uses runtime traces with divergence point detection. DynaFix (Huang et al., 2025) incorporates variable states, control-flow paths, and call stacks. TraceCoder (?) applies fine-grained trace analysis with multi-agent architectures.

2.0.3. 2.3 AUTOMATED PROGRAM REPAIR

Traditional automated program repair uses fault localization to guide patch generation (Dikici & Bilgin, 2025). Recent work has integrated these techniques with LLMs (Li et al., 2024; Kong et al., 2025; Bouzenia et al., 2024).

2.0.4. 2.4 GAP ADDRESSED

This work provides the first controlled comparison of five granularity levels (G0-G4) using the same model, benchmark, and prompt template across all conditions.

3. Methodology

3.0.1. 3.1 EXPERIMENTAL DESIGN

A within-subject design was employed where the same 304 runtime error cases were repaired under all five granularity conditions. This design eliminates case-difficulty confounds.

3.0.2. 3.2 GRANULARITY LEVEL DEFINITIONS

Level	Definition	Example
G0	Pass/fail only	"The code failed the test."
G1	Error type	"The code failed with: IndexError"
G2	Error + message	"IndexError: list index out of range"
G3	Error + line	"IndexError at line 7: list index out of range"
G4	Full stack trace	Complete Python traceback

3.0.3. 3.3 MODEL AND CONFIGURATION

Parameter	Value
Model	CodeLlama-7B-Instruct
Source	meta-llama/CodeLlama-7b-Instruct-hf
Temperature	0 (deterministic)
Max tokens	512
Execution timeout	10 seconds

Temperature 0 was used to ensure reproducibility. The same Self-Debug style prompt template was used across all conditions, with only the error feedback section varying.

3.0.4. 3.4 STATISTICAL ANALYSIS

- **Primary analysis:** One-way ANOVA across five granularity levels
- **Post-hoc comparisons:** Tukey's Honestly Significant Difference (HSD)
- **Paired comparisons:** McNemar's test for G0 vs G3 and G3 vs G4
- **Confidence intervals:** Wilson score intervals for proportions
- **Effect size:** Eta-squared for ANOVA

4. Experimental Setup

4.0.1. 4.1 DATASET

The MBPP benchmark (Austin et al., 2021) was used, consisting of 500 Python programming problems in the test split (task IDs 11-510).

Property	Value
Total problems	500
Split	Test set
Problem type	Single-function Python

4.0.2. 4.2 PROCEDURE

Phase 1 (Error Collection): Code was generated for all 500 MBPP problems using CodeLlama-7B-Instruct. Each solution was executed against test cases and categorized as: PASS, RUNTIME_ERROR, SYNTAX_ERROR, WRONG_OUTPUT, or TIMEOUT.

Phase 2 (Feedback Generation): For each runtime error case, five repair prompts were constructed varying only the error feedback section.

Phase 3 (Repair Evaluation): Repair was attempted under each condition. Success was recorded as binary: 1 if all tests pass, 0 otherwise.

Total repair attempts: 304 cases x 5 levels = 1,520.

4.0.3. 4.3 COMPUTATIONAL RESOURCES

Experiments were conducted on a single NVIDIA A100 GPU. Code generation for H-E1 required approximately 20 minutes. The full H-M1 repair experiment (1,520 attempts) required approximately 40 minutes.

5. Results

5.0.1. 5.1 FOUNDATION: RUNTIME ERROR PREVALENCE (H-E1)

Category	Count	Percentage
Runtime Error	304	60.8%
Syntax Error	193	38.6%
Wrong Output	3	0.6%
Timeout	0	0%
Total Failures	**500**	**100%**

Runtime error prevalence was 60.8% (95% CI: 56.5%-65.0%), exceeding the 30% threshold specified in H-E1. The gate condition was satisfied.

Note: The 0% pass rate for initial code generation indicates that CodeLlama-7B-Instruct did not successfully solve any MBPP problems in this configuration. This does not affect the granularity comparison, which focuses on the distribution of error types among failures.

5.0.2. 5.2 MAIN EFFECT: GRANULARITY AND REPAIR SUCCESS (H-M1)

ANOVA Results:

The ANOVA indicates a statistically significant effect of granularity on repair success. The eta-squared of 0.059 corresponds to a medium effect size by conventional criteria.

Level	Successes	Rate	95% CI
G0	127/304	41.8%	[36.3%, 47.4%]
G1	124/304	40.8%	[35.4%, 46.4%]
G2	56/304	18.4%	[14.4%, 23.2%]
G3	51/304	16.8%	[12.9%, 21.4%]
G4	69/304	22.7%	[18.3%, 27.8%]

Statistic	Value
F-statistic	23.89
p-value	3.5e-19
Eta-squared	0.059

5.0.3. 5.3 POST-HOC PAIRWISE COMPARISONS (TUKEY HSD)

Comparison	Difference	p-value	Significant
G0 vs G1	+1.0pp	0.999	No
G0 vs G2	+23.4pp	5.9e-10	Yes
G0 vs G3	+25.0pp	2.5e-11	Yes
G0 vs G4	+19.1pp	8.3e-07	Yes
G1 vs G2	+22.4pp	3.5e-09	Yes
G1 vs G3	+24.0pp	1.7e-10	Yes
G1 vs G4	+18.1pp	3.6e-06	Yes
G2 vs G3	+1.6pp	0.990	No
G2 vs G4	-4.3pp	0.747	No
G3 vs G4	-5.9pp	0.452	No

The results indicate two clusters: G0 and G1 form a high-success group (41%), while G2, G3, and G4 form a lower-success group (17-23%). Within-cluster differences are not statistically significant.

5.0.4. 5.4 DIRECTIONAL TEST: G0 vs G3 (H-M2)

H-M2 predicted that G3 would outperform G0 by at least 10 percentage points.

Contingency table (paired outcomes):

The difference is in the opposite direction to the hypothesis. G0 outperforms G3 by 25 percentage points. Of the 78 discordant pairs, 77 favor G0 while only 1 favors G3. H-M2 is not supported.

M	c	Value
G0 success rate		41.8%
G3 success rate		16.8%
Difference (G3 - G0)		-25.0pp
95% CI		[-32.0pp, -18.0pp]
McNemar chi-squared		77
p-value		5.23e-22

	G3 Success	G3 Failure
G0 Success	50	77
G0 Failure	1	176

5.0.5. 5.5 NON-MONOTONICITY TEST: G3 vs G4 (H-M3)

H-M3 predicted that G4 would not significantly outperform G3 (difference within 2%).

M ^{Anonymous Organization}	c	Value
G3 success rate		16.8%
G4 success rate		22.7%
Difference (G4 - G3)		+5.9pp
McNemar p-value		4.0e-05

Contingency table:

G4 significantly outperforms G3 ($p < 0.001$), with the difference (5.9pp) exceeding the 2% margin specified in the hypothesis. H-M3 is not supported.

5.0.6. 5.6 SUMMARY OF HYPOTHESIS OUTCOMES

6. Discussion

6.0.1. 6.1 INTERPRETATION OF RESULTS

The data show that for CodeLlama-7B-Instruct on MBPP runtime errors, minimal feedback (G0, G1) yields higher repair success rates than detailed feedback (G2, G3, G4). This pattern was not predicted by the initial hypotheses.

Several possible explanations exist for this pattern, none of which can be definitively established from the current data:

1. **Prompt length:** Detailed feedback increases

	G4 Success	G4 Failure
G3 Success	50	1
G3 Failure	19	234

Hypothesis	Prediction	Result	Status
H-E1	Runtime errors $\geq 30\%$	60.8%	Supported
H-M1	ANOVA $p < 0.05$	$p = 3.5e-19$	Supported
H-M2	$G3 \geq G0 + 10pp$	$G3 = G0 - 25pp$	Not supported
H-M3	$G4 \leq G3 + 2\%$	$G4 = G3 + 5.9\%$	Not supported

prompt length, which may affect model attention. G4 prompts can be substantially longer than G0 prompts due to stack trace content.

1. **Anchoring effects:** Detailed error information may cause the model to focus on the specific error location even when the root cause lies elsewhere.

1. **Model capacity:** At the 7B parameter scale, the model may lack capacity to productively integrate detailed feedback.

These explanations are not mutually exclusive and cannot be distinguished based on the current experiment.

6.0.2. 6.2 CLUSTER PATTERN

The success rates cluster into two groups separated at the G1-G2 boundary. Within the high-success cluster (G0, G1), the difference is 1.0 percentage point (not significant). Within the low-success cluster (G2, G3, G4), differences range from 1.6 to 5.9 percentage points.

6.0.3. 6.3 RELATION TO PRIOR WORK

Self-Debug (Chen et al., 2023) reported 12% improvement on MBPP using approximately G2-level feedback. The current study’s G2 success rate of 18.4% is in a similar range, though direct comparison is limited by differences in experimental setup. The current study extends prior work by including the G0 condition, which was not systematically evaluated in Self-Debug.

Haque et al. (2025) observed that execution traces provide limited improvement for LLM repair unless prompts are optimized for LLMs. The current results are consistent with this observation.

6.0.4. 6.4 LIMITATIONS

The following limitations constrain the generalizability of these findings:

L1: Single Model. Results are specific to CodeLlama-7B-Instruct. Larger models (13B, 34B, 70B) may show different patterns. No claims are made about optimal granularity for larger models.

L2: Single Benchmark. MBPP consists of relatively simple single-function problems. Results may differ for more complex debugging tasks.

L3: Single Prompt Template. The Self-Debug template was used throughout. Template-granularity interactions are possible but were not tested.

L4: Single Repair Attempt. Only single-turn repair was evaluated. Multi-turn iterative repair may show different patterns.

L5: Deterministic Generation. Temperature 0 was used for reproducibility. Stochastic sampling may yield different results.

L6: Runtime Errors Only. The study focused on runtime errors (60.8% of failures). Silent logic errors producing wrong outputs may respond differently to granularity variations.

L7: Zero Initial Pass Rate. CodeLlama-7B-Instruct did not successfully solve any MBPP problems in this configuration before repair. This unusual baseline may affect generalizability.

6.0.5. 6.5 IMPLICATIONS

For CodeLlama-7B-Instruct on MBPP-style problems, minimal error feedback appears more effective than detailed feedback for single-turn repair. Whether this finding transfers to other models, tasks, or configurations requires additional empirical investigation.

7. Conclusion

This study compared five levels of error feedback granularity for LLM code repair using CodeLlama-7B-Instruct on 304 MBPP runtime error cases. The main findings are:

- Granularity has a statistically significant effect on repair success (ANOVA $F=23.89$, $p=3.5e-19$, $\eta^2=0.059$).
- For this model and task, minimal feedback (G0: 41.8%, G1: 40.8%) yields higher repair success than detailed feedback (G2: 18.4%, G3: 16.8%, G4: 22.7%).
- The initial hypotheses predicting optimal intermediate granularity were not supported. The

data show G0 outperforming G3 by 25 percentage points.

- These results are specific to CodeLlama-7B-Instruct on MBPP and should not be extrapolated to larger models or different tasks without further empirical validation.

Future work should investigate whether the observed pattern holds for larger models, different prompt templates, and more complex debugging tasks.

References

- Austin, J., Odena, A., Nye, M., Bosma, M., Michalewski, H., Dohan, D., Jiang, E., Cai, C., Terry, M., Le, Q., and Sutton, C. Program synthesis with large language models. *arXiv preprint arXiv:2108.07732*, 2021.
- Bouzenia, I., Ding, Y., Pei, K., Ray, B., and Pradel, M. Tracefixer: Execution trace-driven program repair. *arXiv preprint arXiv:2304.12743*, 2023.
- Bouzenia, I. et al. Repairagent: An autonomous, llm-based agent for program repair. *arXiv preprint*, 2024.
- Chen, X., Lin, M., Schärli, N., and Zhou, D. Teaching large language models to self-debug. In *International Conference on Learning Representations*, 2023.
- Dikici, A. and Bilgin, Z. Automated program repair: A survey. *arXiv preprint*, 2025.
- Gehring, J., Zheng, K., Copet, J., Mella, V., Cohen, T., and Synnaeve, G. Rlef: Grounding code llms in execution feedback with reinforcement learning. In *International Conference on Machine Learning*, 2024.
- Haque, M., Babkin, P., Farmahinifarahani, F., and Veloso, M. Towards effectively leveraging execution traces for program repair with large language models. In *Proceedings of the 4th International Workshop on Knowledge-Augmented Methods for Natural Language Processing*, 2025.
- Huang, Y. et al. Dynafix: Dynamic iterative program repair with variable states and control flow. *arXiv preprint*, 2025.
- Jiang, N., Li, X., Wang, S., Zhou, Q., Hossain, S. B., Ray, B., Kumar, V., Ma, X., and Deoras, A. Ledex: Training llms to better self-debug and explain code. In *Advances in Neural Information Processing Systems*, 2024.

Li, J., Le, H., Zhou, Y., Xiong, C., Savarese, S., and Sahoo, D. Codetree: Agent-guided tree search for code generation with large language models. In *Proceedings of the 2025 Conference of the North American Chapter of the Association for Computational Linguistics*, 2024.

Rozière, B., Gehring, J., Gloeckle, F., Sootla, S., Gat, I., Tan, X., Adi, Y., Liu, J., Remez, T., Rapin, J., et al. Code llama: Open foundation models for code. *arXiv preprint arXiv:2308.12950*, 2023.